Microcontroller-Based Voltage Protector

FSMB Recruitment 2026, Phase 1 | Embedded System Engineer | Ehsan Mahmud Sourav

What I built and how I approached it

I built an Arduino Uno based protector that monitors the supply voltage and load current and switches a relay off when a fault occurs. The whole project is simulated in Proteus 8 Professional. Since Proteus has no ready-made voltage or current sensor, I built the sensing circuits from standard components. The relay connects and disconnects the supply to the appliance. A green LED shows the NORMAL state, a red LED shows WAITING or FAULT, a buzzer sounds on a fault, and a 16x2 LCD shows what the system is doing. A reset button clears the overcurrent faults only, because voltage fluctuations on the power grid are usually temporary external events. Three menu buttons let the user change settings directly on the device. The load stays OFF at boot, so after a reset or a power glitch the appliance gets no supply until the mains is checked. The code is non-blocking & modular, so serial commands and buttons keep working while protection is running. Settings are saved in EEPROM and kept after power-off, and the device can also be monitored, adjusted and controlled from a computer.

How the protection works

Every loop, measure() samples both channels for exactly one 20 ms mains cycle and computes the RMS voltage, RMS current and peak voltage (the DC bias is subtracted first). Then protect() checks the faults in priority order:

Fault	Trigger	Response	Recovery
Surge	> 270 V RMS or > 400 V peak	instant	automatic
Over-voltage	> OVP (default 250 V)	5 cycles, ~100 ms	automatic
Under-voltage	< UVP (default 180 V)	25 cycles, ~500 ms	automatic
Short circuit	> 30 A	instant	latched
Heavy overload	> 15 A	5 cycles	latched
Overload	> OCP (default 10 A)	inverse-time: adds (I/OCP)^2 each cycle, trips at 250	latched

Voltage faults (surge, over-voltage, under-voltage) reconnect automatically once the supply has stayed between 190 and 240 V for the reconnect delay. Overcurrent faults are latched and need the reset button or RELAY ON to reconnect.

Settings and computer communication

The thresholds OVP (230-280 V), UVP (100-210 V), OCP (1-25 A) and the reconnect delay (1-600 s) can be changed in two ways.

On the device: SELECT opens a menu on the LCD and steps through the four settings. UP/DOWN change the value (holding the key for more than 2 s on the delay setting makes it jump by 10 s). The menu saves and exits after 10 s without a key press.

From a computer: a 9600 baud text protocol (GET STATUS, SET OVP/UVP/OCP/DELAY n, RELAY ON/OFF, STREAM ON/OFF, GET LOG, CLEAR LOG) with replies. Out of range values return ERR RANGE and unknown commands return ERR CMD. Every trip is also pushed to the PC automatically, and the last 5 trips are kept in a log. For the PC side I wrote a small Python (pyserial) tool, protector_cli.py, which talks to the Proteus COMPIM port through a VSPE virtual COM pair.

What I tested

I ran all eight command types (GET STATUS, the four SET commands, RELAY OFF/ON, GET LOG) from the Python tool through Proteus and got the expected results. Getting there took hours of attempts. The problems I faced were:

1. Scaling every section of the circuit (solved by trial and error).
2. Creating voltage faults, because values cannot be changed while the simulation is running (solved by adding several supplies with SPDT switches). These sources exist only to create demo faults, so use one at a time: turn one off, wait until the state goes back to normal, then connect the next (see the weaknesses for two together).
3. No data reached the virtual terminal and COMPIM when I connected both at the same time (connect one at a time).
4. COMPIM TXD and RXD are named with reference to the Arduino's TXD and RXD (connect TXD to TXD and RXD to RXD between the Arduino and COMPIM).
5. Writing the pyserial tool, which I learned today.

What is not done or not good enough yet

Not tested enough: first-command boot race condition (unconfirmed). The very first command sent after boot may still occasionally return ERR CMD if it arrives during the 1-second splash screen. I have not added the debug function (printBadCmd) that would capture this yet.

Known weaknesses: the reconnect window (190-240 V) is hard-coded and independent of the custom OVP/UVP thresholds. Settings changed from the on-device menu are saved to EEPROM only when the menu is exited (final SELECT or the 10-second timeout), while serial SET commands save immediately. The processCmd function uses the Arduino String class, which can fragment memory on AVR microcontrollers over long run times.

Demo fault setup: it uses extra sources behind SPDT switches, which is a workaround, not a real grid. Two voltage sources together is not practical, since a real supply cannot be at two voltages at once. With both current switches on, the status does not change by itself although the measured current stays accurate; the short circuit fault appears only after the reset button is pressed.

Safety: this is a simulation. A real mains product needs galvanic isolation on the sensing, a fuse or breaker, and a relay and shunt rated for the load. Software protection alone is not enough.